EXCEPTIONS

After completing this module, the students will be able to:

- Understand situations when exceptions are thrown
- Understand the concept of exceptions handling
- Identify situations when exceptions are thrown
- Add code to handle and throw exceptions
- Write his/her own exception classes

After completing this module, the students will be able to:

- Pass additional information in exceptions
- Handle file-related exceptions
- Throw exceptions in constructors
- Use exceptions in real programs
- Simplify exception throwing

How to get into trouble?

There is no code free of errors.

- One of Murphy's laws even states that

"An error-free code is actually only one that has been insufficiently tested"

Take a look at the example →

```
#include <iostream>
using namespace std;
int main(void) {
    float a, b;
    cin >> a;
    cin >> b;
    cout << a / b << endl;
    return 0;

}
```

Compile and run the program and enter “0” (zero) as the second value. What happens then?

Take a look at the example →

```
#include <iostream>
using namespace std;
int main(void) {
      float a, b;
      while(cin >> a) {
      cin >> b;
      cout << a / b << endl;
}
     return 0; }
```

HOW TO GET OUT OF TROUBLE ?

- When something goes wrong in a program, an exception arises.
- Some exceptions arise automatically, completely out of our control, while others may be created at our request.

HOW TO GET OUT OF TROUBLE ? (CONTD.)

- The part of the code that may cause problems needs to be marked
- When an exception arises, the execution of the block is terminated

WHAT IS EXCEPTIONS?

- An exception is a problem that arises during the execution of a program.
- A C++ exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

WHAT IS EXCEPTIONS?

- Exceptions
  - Indicate problems that occur during a program's execution
  - Occur infrequently
- Exception handling
  - Can resolve exceptions
    - Allow a program to continue executing or
    - Notify the user of the problem and
    - Terminate the program in a controlled manner
  - Makes programs robust and fault-tolerant

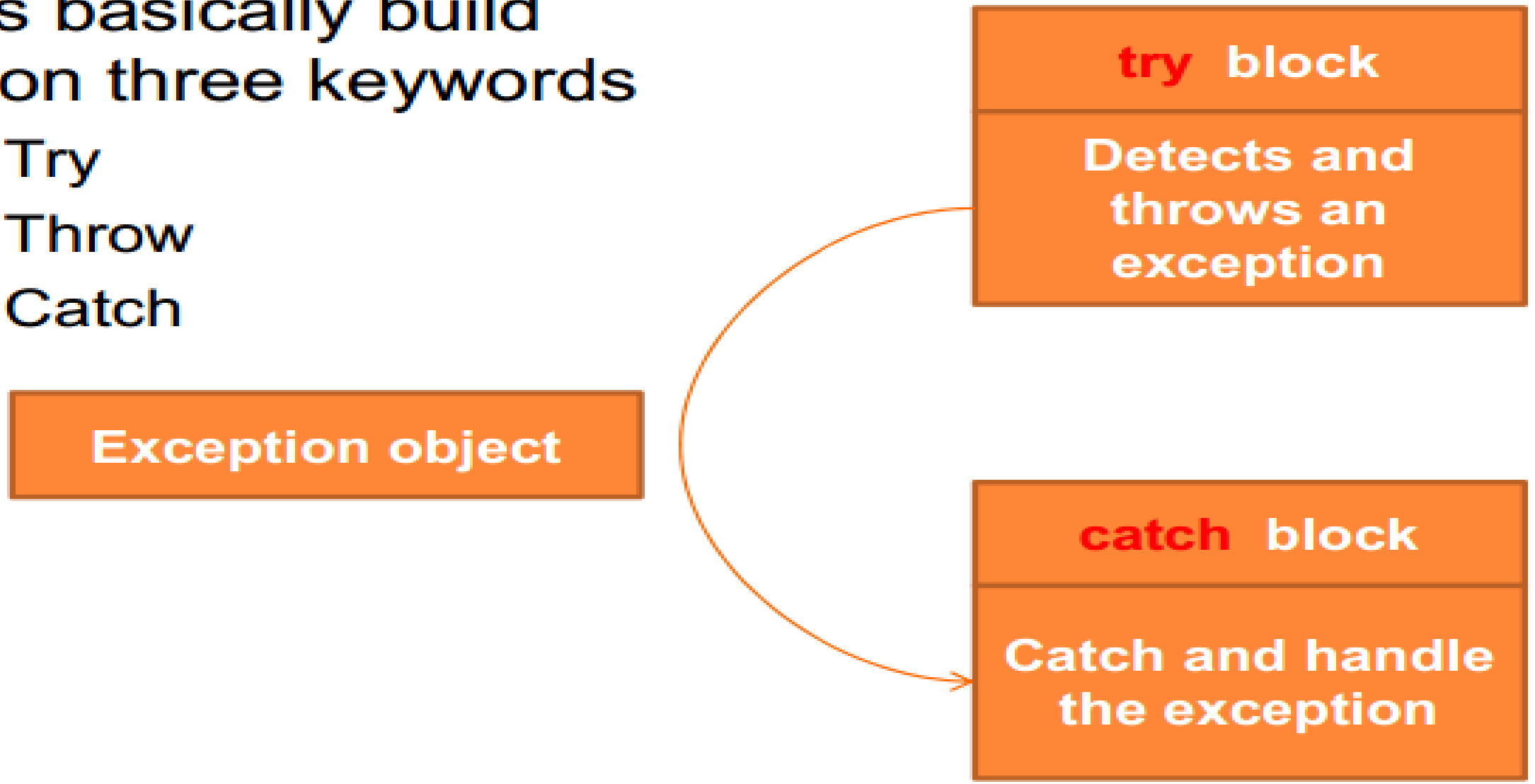
WHY EXCEPTION HANDLING?

- Following are main advantages of exception handling over traditional error handling.

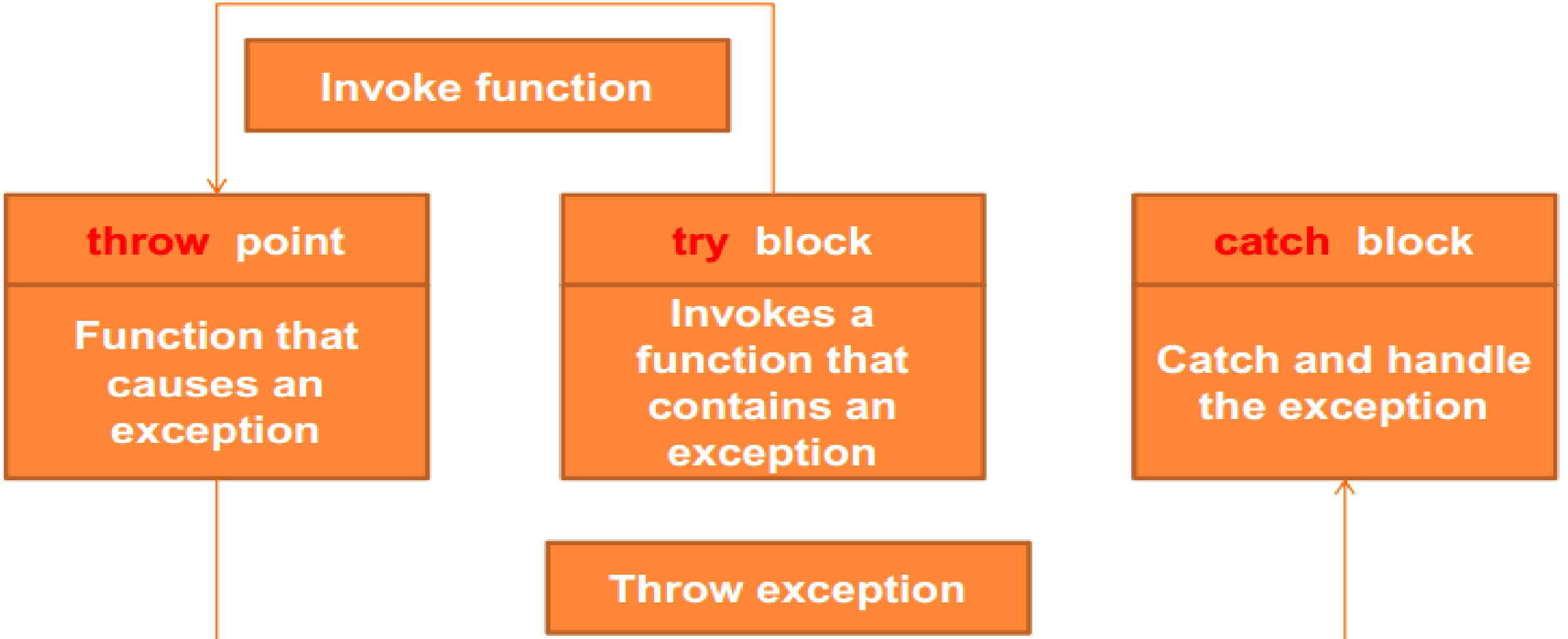
1) *Separation of Error Handling code from Normal Code*
2) *Functions/Methods can handle any exceptions they choose*
3) *Grouping of Error Types*

Exception Handling Mechanism

- It is basically build upon three keywords
  - Try
  - Throw
  - Catch



Exception Handling Mechanism (Cont…)



WHAT IS EXCEPTIONS?

- Exceptions provide a way to transfer control from one part of a program to another. C++ exception handling is built upon three keywords: try, catch, and throw.
- **throw** − A program throws an exception when a problem shows up. This is done using a **throw** keyword.
- **catch** − A program catches an exception with an exception handler at the place in a program where you want to handle the problem.
  The **catch** keyword indicates the catching of an exception.
- **try** − A **try** block identifies a block of code for which particular exceptions will be activated. It's followed by one or more catch blocks.

THROW, CATCH, TRY

- **throw** − A program throws an exception when a problem shows up. This is done using a **throw** keyword.
- **catch** − A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The **catch** keyword indicates the catching of an exception.
- **try** − A **try** block identifies a block of code for which particular exceptions will be activated. It's followed by one or more catch blocks.

THROW, CATCH, TRY

- Assuming a block will raise an exception, a method catches an exception using a combination of the **try** and **catch** keywords.
- A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as protected code, and the syntax for using try/catch as follows –

```
try {
   // protected code
} catch( ExceptionName e1 ) {
   // catch block
} catch( ExceptionName e2 ) {
   // catch block
} catch( ExceptionName eN ) {
   // catch block
}
```

THROWING EXCEPTIONS

- Exceptions can be thrown anywhere within a code block using throw statement.
- The operand of the throw statement determines a type for the exception and can be any expression and the type of the result of the expression determines the type of exception thrown.

- If you write it this way:

THROWING EXCEPTIONS

- Following is an example of throwing an exception when dividing by zero condition occurs −

```
double division(int a, int b) {
   if( b == 0 ) {
      throw "Division by zero
condition!";
   }
   return (a/b);
}
```

CATCHING EXCEPTIONS

- The catch block following the try block catches any exception.
- You can specify what type of exception you want to catch and this is determined by the exception declaration that appears in parentheses following the keyword catch.

```
try {
   // protected code
} catch( ExceptionName e1 ) {
   // catch block
} catch( ExceptionName e2 ) {
   // catch block
} catch( ExceptionName eN ) {
   // catch block
}
```

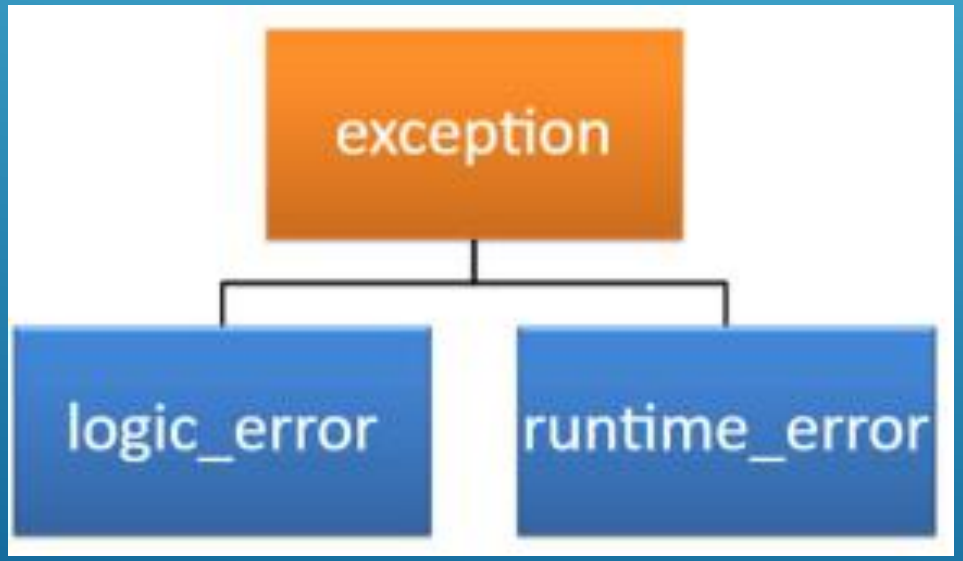
CATCHING EXCEPTIONS

- Above code will catch an exception of ExceptionName type.
- If we want to specify that a catch block should handle any type of exception that is thrown in a try block, we must put an ellipsis, ..., between the parentheses enclosing the exception declaration as follows −

```
try {
   // protected code
} catch( . . .) {
 // code to handle any exception
}
```

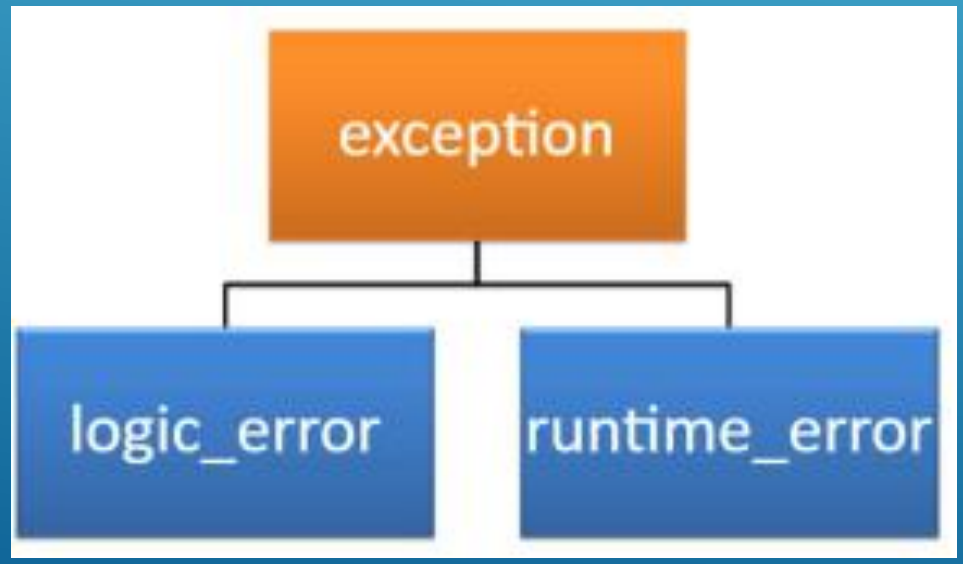
THE HIERARCHY IS HERE →

- The top-most class, named *exception*, is a base for all exceptions, the predefined ones as well as these that are user-defined.
- Generally, objects of this class are neither created nor thrown, and the class is only used to define some behaviours and properties common to all exceptions.



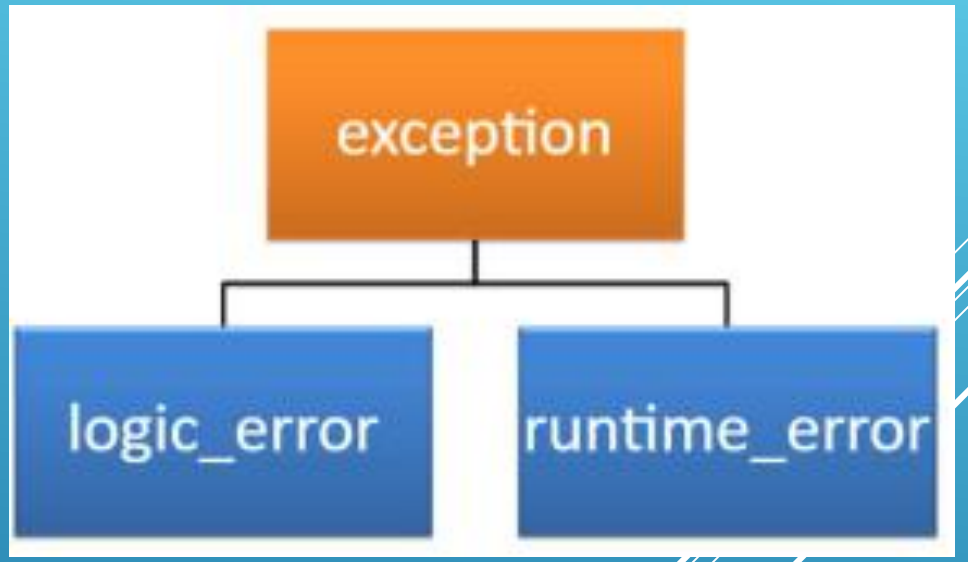
THE HIERARCHY IS HERE →

- The top-most class, named *exception*, is a base for all exceptions, the predefined ones as well as these that are user-defined.
- Generally, objects of this class are neither created nor thrown, and the class is only used to define some behaviours and properties common to all exceptions.



THE HIERARCHY IS HERE →

- Two different classes are derived from the exception class. The first, named logic_error, is intended to represent exceptions connected to program logic i.e. the algorithm, its implementation, data validity and cohesion.
- The second class, named runtime_error, is used to identify exceptions thrown due to “unexpected” accidents like a lack of memory.



ANATOMY OF AN EXCEPTION OBJECT

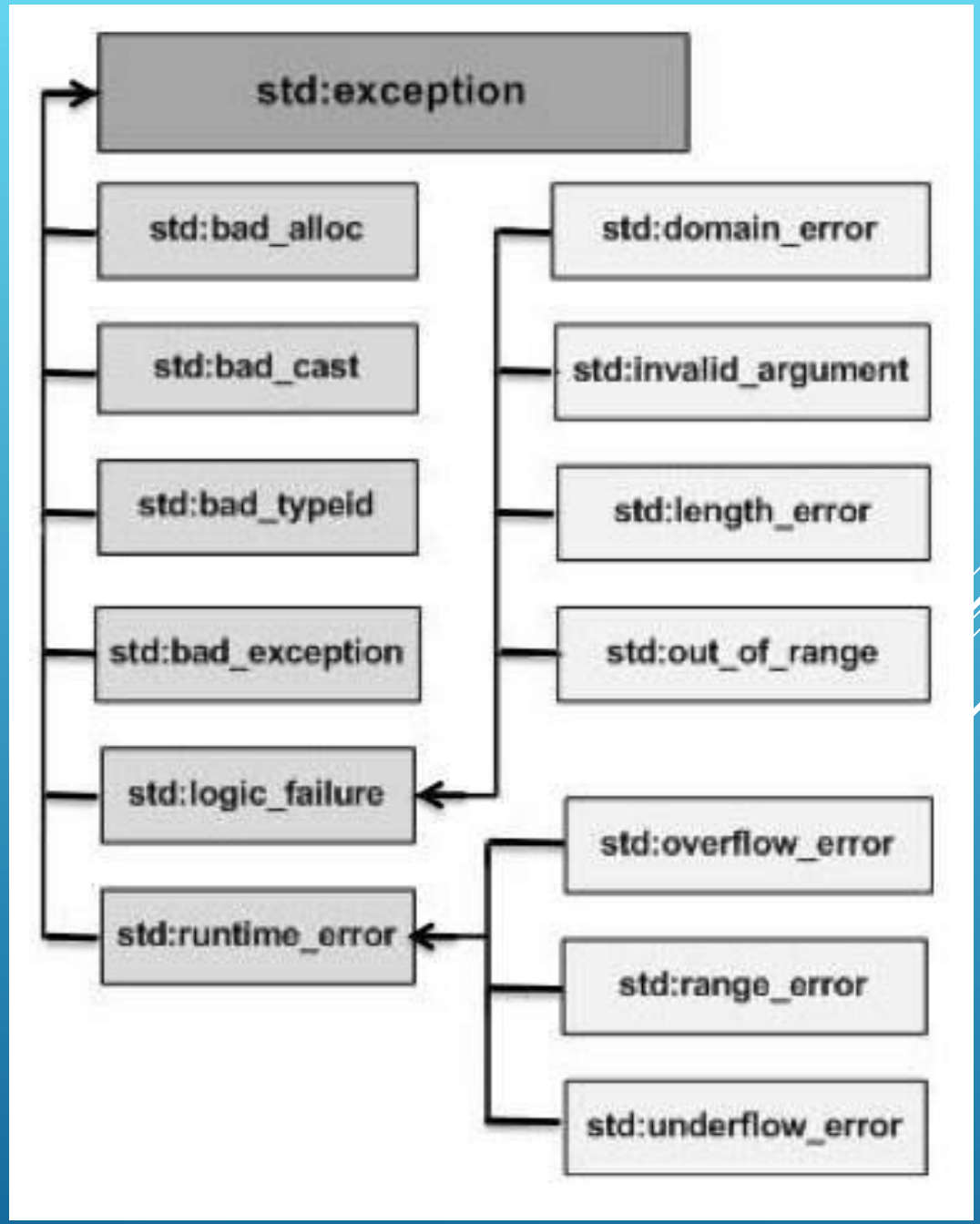
- The exception class is very modest. In fact it defines only three components:
  - a constructor
  - a virtual destructor, originally empty
  - a virtual function called what which returns the C-style string

```
virtual char* what()
```

  - The what function provides a text (more or less verbose) describing the nature and cause of the exception.

C++ STANDARD EXCEPTIONS

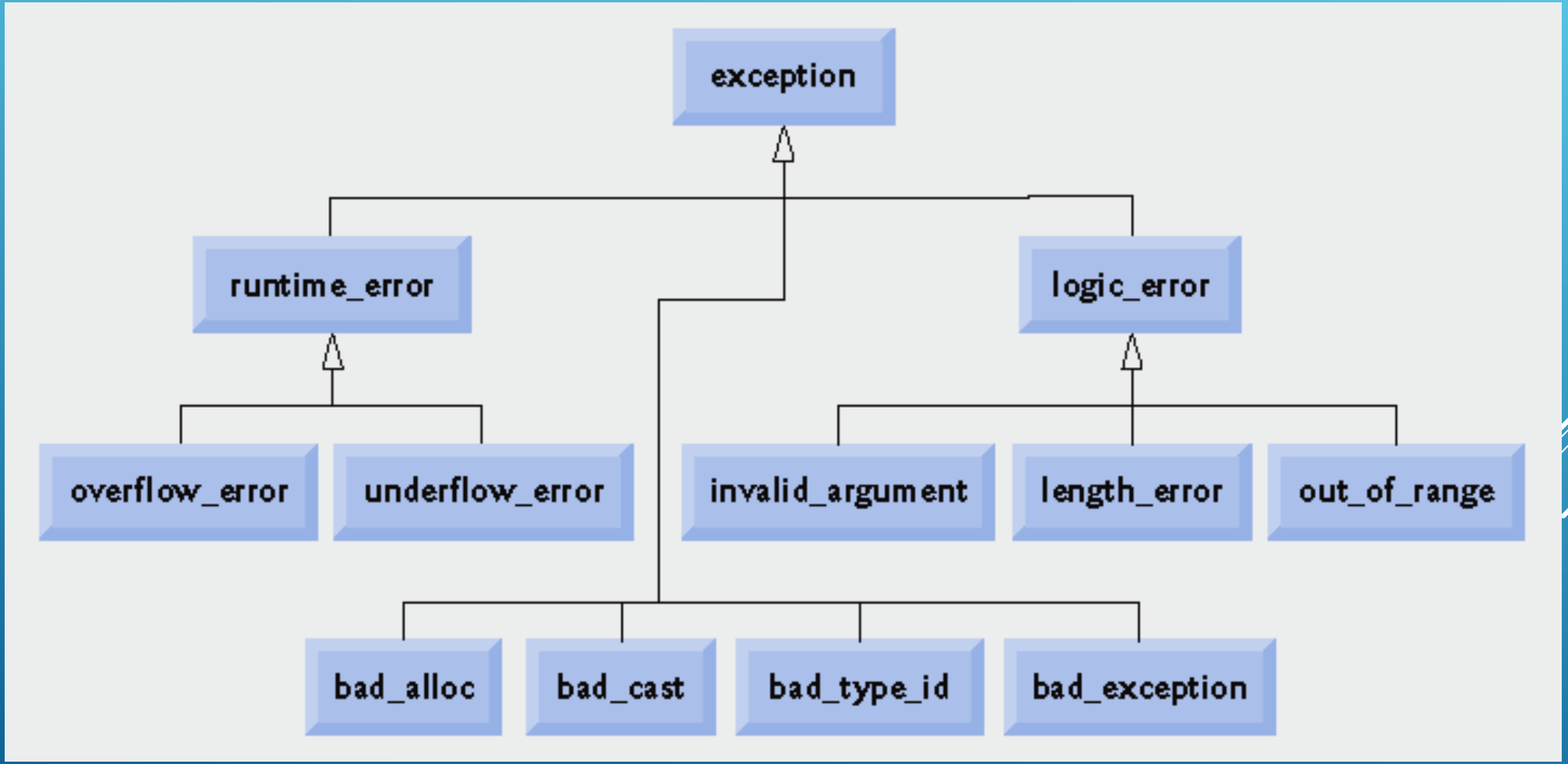
- C++ provides a list of standard exceptions defined in <exception> which we can use in our programs. These are arranged in a parent-child class hierarchy shown below −



STANDARD LIBRARY EXCEPTION HIERARCHY

- Base-class **`exception`**
  - Contains **`virtual`** function **`what`** for storing error messages
- Exception classes derived from **`exception`**
  - **`bad_alloc`** – thrown by **`new`**
  - **`bad_cast`** – thrown by **`dynamic_cast`**
  - **`bad_typeid`** – thrown by **`typeid`**
  - **`bad_exception`** – thrown by **`unexpected`**
    - Instead of terminating the program or calling the function specified by **`set_unexpected`**
    - Used only if **`bad_exception`** is in the function's **`throw`** list

STANDARD LIBRARY EXCEPTION CLASSES.



THE ‘EXPLICIT’ KEYWORD

- The explicit keyword may be placed in front of a class’s constructor declaration.
- It protects the constructor from being used in any context requiring the use of implicit conversions.
- This constructor may only be used in an explicit way; hence, the keyword is used when a programmer wants to avoid any side effects that may result from the reckless use of the selected constructors, or indeed of all constructors.

Take a look at the example →

```
class A {
public:
    explicit A(int) {}
};
class B {
public:
   B(int) {}
};
int main(void) {
    A a = 1; // compilation error!
    B b = 1;
    return 0;
}
```

THE ‘EXCEPTION’ CLASS

- The exception class is a base (or a root) for all other predefined exceptions.
- It contains a function called what, which is designed to provide a pointer to the so-called “C”-style string (a character sequence terminated with a null character) describing the nature of the exception.

THE ‘EXCEPTION’ CLASS

- The example program here → illustrates the functioning of that mechanism.

```
#include <iostream>
#include <exception>
using namespace std;
    class A {
    public:
    virtual void f(void) {}
};
```

```
class AA : public A {
    public:
    void aa(void) {};
};

int main(void) {
    A a;
    try {
        dynamic_cast<AA &>(a).aa();
    }   catch (exception ex) {
        cout << "[" << ex.what() << "]" << endl;
    }
    return 0;
    }
```

THE ‘LOGIC_ERROR’ CLASS

- The *logic_error* class is directly derived from the *exception* class.

```
class logic_error : public exception {
    public:
    explicit logic_error (const string& what_arg);
}
```

- It’s designed to represent all the exceptions caused by a violation of the rules imposed by the logic of the algorithm/program

DEFINE NEW EXCEPTIONS

- You can define your own exceptions by inheriting and overriding exception class functionality. Following is the example, which shows how you can use std::exception class to implement your own exception in standard way −

```
#include <iostream>
#include <exception>
using namespace std;

struct MyException : public exception {
   const char * what () const throw () {
      return "C++ Exception";
   }
};

int main() {
   try {
      throw MyException();
   } catch(MyException& e) {
      std::cout << "MyException caught" << std::endl;
      std::cout << e.what() << std::endl;
   } catch(std::exception& e) {
      //Other errors
   }
}
```

Here, **what()** is a public method provided by exception class and it has been overridden by all the child exception classes. This returns the cause of an exception.

END…